# AI/ML IN SECURITY

## CREDIT CARD FRAUD DETECTION SYSTEM


By -

Alankrita Vasishth – 590022944

Bhavya Singh – 590022993

Rachit Kumar – 590023269

Vatsal agarwal– 590023223

# Problem Statement – The Financial Fraud Crisis

- **The Global Threat Landscape:** As digital transaction volumes hit record highs, traditional security measures are failing to keep pace with sophisticated "Synthetic Identity" and "Card-Not-Present" fraud.

- **Limitations of Legacy Systems:** Static rule-based engines (e.g., "Flag if amount > $5000") are too predictable. Scammers easily bypass these thresholds by staying just below them.

- **The Cybersecurity Gap:** Modern fraud requires a "Pattern-First" approach. Without automated detection, banks face massive capital loss, legal liabilities, and a breakdown in consumer trust.

- **Legal Necessity:** Regulatory bodies (RBI/SEBI) now mandate "Reasonable Security Practices." Failing to detect preventable fraud can lead to heavy penalties under the IT Act.

# Project Objectives & System Architecture

- **End-to-End Automation:** To develop a high-speed analytical pipeline that ingests raw transaction logs and outputs actionable risk scores.

- **ML-Powered Accuracy:** Utilizing Ensemble Learning (Random Forest) to detect non-linear fraud patterns that human auditors would miss.

- **The Architecture:**
    - **Frontend:** Minimalist dashboard for data upload and visualization.
    - **Backend (Python/Flask):** The central hub for file handling, data cleaning, and model routing.
    - **ML Layer:** A pre-trained classifier capable of millisecond inference.

- **Scalability:** Designed to handle thousands of transactions in a single batch processing request.

# Exploratory Data Analysis (EDA) & Preprocessing

- **Feature Correlation:** Analyzing how transaction magnitude (Amount) and merchant categories correlate with fraudulent behavior.

- **Handling Class Imbalance:** Fraud is rare (often <1% of data). Our EDA process ensures the model doesn't become "biased" toward legitimate transactions.

- **The Data Pipeline:**
  - **Cleaning:** Removing corrupted entries and handling missing values via Pandas.
  - **Scaling:** Normalizing numerical features so high-value transactions don't skew the model's perception.
  - **Encoding:** Converting categorical metadata into mathematical vectors for model ingestion.

# Machine Learning Engine: Random Forest Classifier

- **Ensemble Intelligence:** Random Forest operates by constructing a multitude of decision trees. It outputs the class that is the "mode" of the individual trees, providing superior accuracy over single-tree models.
- **Probabilistic Scoring:** Instead of a binary "Yes/No," the system utilizes predict_proba. This provides a percentage-based confidence level, which is essential for forensic investigation.
- **Resistance to Overfitting:** By using bagging and feature randomness, the model stays robust even when scammers change their tactics slightly.
- **Performance Metrics:** The model focuses on the "F1-Score"—balancing the need to catch thieves (Recall) with the need to avoid annoying real customers (Precision).

# Technical Breakdown: The Analysis API (app.py)

• **Request Handling:** The @app.route('/analyze') function manages the POST request, ensuring the file is valid and readable.

• **Data Orchestration:** Converting the raw CSV buffer into a structured Pandas DataFrame for high-speed feature extraction.

• **Model Inference:**

```
# Load and Preprocess
df = pd.read_csv(file)
features = df.drop(columns=['id'])

# Generate Probabilities
probabilities = model.predict_proba(features)[:, 1]
predictions = model.predict(features)
```

# Technical Breakdown: Risk & Summary Logic

- **Human-Centric Categorization:** The backend translates raw percentages into a "Traffic Light" risk system:
  - **HIGH (>80%):** Immediate red flag; likely unauthorized access or stolen credentials.
  - **MEDIUM (50-80%):** Anomalous behavior; requires secondary authentication or manual review.
  - **LOW (<50%):** Verified pattern; standard transaction.

- **Strategic Summary Logic:**

```
# Calculating executive stats
summary = {
    "total": len(df),
    "fraud": int(sum(predictions)),
    "normal": int(len(df) - sum(predictions))
}
```

- **JSON Response:** The data is sent as a structured object, making it easy for any UI to display results dynamically.

# Cybersecurity, Privacy & DPDP Compliance

- **Data Minimization:** Adhering to the principle that only the minimum data required for a specific purpose (fraud detection) should be processed.
- **Volatile Processing:** Data is handled in RAM and not permanently written to the server's disk, significantly reducing the "Attack Surface" for data breaches.
- **IT Act & DPDP 2023:** The system is designed to provide "Reasonable Security" under Section 43A of the IT Act while respecting the consent and privacy mandates of the new Digital Personal Data Protection Act.
- **Input Sanitization:** Strict validation of CSV structures prevents common exploits like CSV Injection or Malicious File Uploads.

# Project Impact, Challenges & Future Horizons

- **Operational Impact:** Drastically reduces the time-to-detection, allowing financial institutions to block fraudulent funds before they are withdrawn.

- **Key Challenges:**
  - **Model Drift:** The ongoing battle of retraining the AI as new scam patterns emerge.
  - **Latency:** Maintaining millisecond response times as the transaction volume scales.

- **The Future:**
  - **Explainable AI (XAI):** Integrating SHAP values to explain *why* the AI flagged a transaction.
  - **Real-time Integration:** Moving toward a Live-API model for instant, "at-the-swipe" protection.

# THANK YOU!

A

PROJECT CODE FILE

On

# Credit Card Fraud Detection System

## Elements of AIML

(CSAI2018_3)

School of Computer Science, UPES, Dehradun.


B.TECH. - II Semester

**Jan. – May (2026)**

**Submitted to:**
Dr. Vineet Jaiswal
Assistant Professor
SoCS, UPES

**Submitted by:**

| Name | SAP ID |
|---|---|
| Alankrita Vasishth | 590022944 |
| Bhavya Singh | 590022993 |
| Rachit Kumar | 590023269 |
| Vatsal Agarwal | 590023223 |

# INDEX

# METHODOLOGY

**(1) Dataset Used :**

- The project uses two datasets:
  - creditcard.csv → Used as the training dataset
  - test_transactions.csv → Used as the testing dataset
- The training dataset contains historical transaction data with labeled classes (fraud/normal)
- The testing dataset is used to simulate real-time transaction analysis
- Features include Time, Amount, and V1–V28 (anonymized variables)

**(2) Data Preprocessing :**

- Removed the target column (Class) from input features
- Applied StandardScaler to normalize feature values
- Ensured both datasets have the same structure and feature order
- Handled missing or unnecessary columns during testing

**(3) Backend Implementation :**

- The backend is implemented using app.py
- Built using Flask framework and Python
- Responsibilities:
  - Load and train the machine learning model
  - Process uploaded test dataset
  - Perform predictions using Logistic Regression
  - Return results in JSON format
- Model outputs:
  - Fraud prediction (0 or 1)
  - Fraud probability
  - Risk level (Low / Medium / High)

**(4) Frontend Implementation :**

- The frontend is implemented using index.html
- Built using HTML, CSS, and JavaScript
- Responsibilities:
  - Provide user interface for file upload
  - Send data to backend using HTTP request
  - Display results in dashboard format
  - Show summary (total, fraud, normal transactions)
  - Present transaction details in a table with risk indicators

# DATASET USED

**(1) creditcard.csv (Training Dataset)**

- Contains historical transaction data
- Used to train the machine learning model
- Highly imbalanced dataset (fraud cases are very few)
- Total features: 30+ columns

**Key Features :**

- **Time** → Time elapsed since first transaction
- **Amount** → Transaction amount
- **V1 to V28** → Anonymized features (PCA transformed for privacy)
- **Class** → Target variable
  - 0 → Normal transaction
  - 1 → Fraud transaction

**(2) test_transactions.csv (Testing Dataset)**

- Used for **real-time prediction (input by user)**
- Simulates new/unseen transactions
- Same structure as training dataset (except Class may be absent)

**Key Features :**

- **Time** → Transaction time
- **Amount** → Transaction value
- **V1 to V28** → Input features for prediction
- **Class** → Used only for testing accuracy (not required)

# BACKEND IMPLIMENTATION

```
import pandas as pd
import numpy as np
from flask import Flask, request, jsonify, render_template
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
import os

app = Flask(__name__)

# Global variables (learned in Unit 2: Collections and Functions)
model = None
scaler = None

def train_model():
    """Trains the model using logic from your Experiment 7 and 10."""
    global model, scaler

    # Check if file exists (Unit 3: File Handling)
    if not os.path.exists("creditcard.csv"):
        print("Error: creditcard.csv not found in the project folder.")
        return

    print("Loading dataset for training...")
    # Load data using Pandas (CO5: Data Manipulation)
    df = pd.read_csv("creditcard.csv")

    # Features (X) are all columns except 'Class'
    # Target (y) is the 'Class' column (0=Normal, 1=Fraud)
```

```
    X = df.drop("Class", axis=1)
    y = df["Class"]

    # Scaling (Experiment 7: Preprocessing)
    # StandardScaler makes sure the 'Amount' column doesn't overpower others
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Training (Experiment 6: Elements of AIML)
    print("Training Logistic Regression model...")
    model = LogisticRegression(max_iter=1000)
    model.fit(X_scaled, y)
    print("Model is ready! Open http://127.0.0.1:5000 in your browser.")

@app.route("/")
def index():
    """Displays your HTML dashboard."""
    return render_template("index.html")

@app.route("/analyze", methods=["POST"])
def analyze():
    """Receives your test CSV and returns JSON for the HTML dashboard."""
    if model is None:
        return jsonify({"error": "Model not trained yet."}), 500

    # Get the file from the request
    file = request.files.get("file")
    if not file:
        return jsonify({"error": "No file uploaded."}), 400
```

```
try:
    # Read the test data
    df_test = pd.read_csv(file)

    # We ignore the 'Class' column if it exists in the test file
    X_input = df_test.drop("Class", axis=1) if "Class" in df_test.columns else df_test

    # Scale using the training scaler
    X_test_scaled = scaler.transform(X_input)

    # Get Predictions and Probabilities
    predictions = model.predict(X_test_scaled)
    probabilities = model.predict_proba(X_test_scaled)[:, 1]

    results = []
    for i in range(len(df_test)):
        prob = float(probabilities[i])

        # Logic for Risk Level in your HTML
        if prob >= 0.7:
            risk = "HIGH"
        elif prob >= 0.3:
            risk = "MEDIUM"
        else:
            risk = "LOW"

        results.append({
            "transaction_id": i + 1,
            "amount": round(float(df_test["Amount"].iloc[i]), 2),
```

```
            "time": int(df_test["Time"].iloc[i]),
            "fraud_probability": round(prob * 100, 1),
            "prediction": int(predictions[i]),
            "risk_level": risk
        })

    # Summary for the top dashboard cards
    summary = {
        "total": len(results),
        "fraud_count": int(sum(predictions)),
        "normal_count": len(results) - int(sum(predictions)),
    }

    return jsonify({"results": results, "summary": summary})

  except Exception as e:
    return jsonify({"error": f"Analysis failed: {str(e)}"}), 500

if __name__ == "__main__":
  train_model()
  app.run(debug=True)
```

```
PS C:\Users\Alankrita Vasishth\Desktop\AI ML LAB Codes\CreditCardProject> python -u app.py
Model trained successfully!
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
Model trained successfully!
 * Debugger is active!
 * Debugger PIN: 647-587-036
```

**OUTPUT**

# FRONTEND DASHBOARD

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fraud Detection System</title>
  <style>
    /* Color Palette and Variables */
    :root {
      --primary-dark: #2c3e50;
      --accent-blue: #3498db;
      --danger: #e74c3c;    /* Red for High Risk */
      --warning: #f39c12;   /* Orange for Medium Risk */
      --success: #27ae60;   /* Green for Low Risk */
      --light-bg: #f8f9fa;
    }

    body {
      font-family: 'Segoe UI', Arial, sans-serif;
      background-color: #ecf0f1;
      margin: 0;
      padding: 40px;
      color: var(--primary-dark);
    }

    .container {
      max-width: 1000px;
```

```
    margin: 0 auto;
    background: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.1);
}

h2 { border-bottom: 2px solid var(--light-bg); padding-bottom: 10px; }

/* Upload Section */
.upload-card {
    background: var(--light-bg);
    padding: 25px;
    border-radius: 8px;
    text-align: center;
    border: 2px dashed #bdc3c7;
    margin-bottom: 30px;
}

button {
    background-color: var(--accent-blue);
    color: white;
    border: none;
    padding: 12px 24px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: 600;
    transition: background 0.3s;
}
```

```
button:hover { background-color: #2980b9; }

/* Summary Stats */
.summary-container {
    display: flex;
    gap: 20px;
    margin-bottom: 30px;
}

.stat-card {
    flex: 1;
    padding: 20px;
    border-radius: 8px;
    text-align: center;
    background: white;
    border: 1px solid #eee;
}

.stat-card h3 { margin: 0; font-size: 14px; text-transform: uppercase; color: #7f8c8d; }
.stat-card span { font-size: 28px; font-weight: bold; display: block; margin-top: 5px; }

#fraud { color: var(--danger); }
#normal { color: var(--success); }

/* Table Styling */
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
}
```

```
    th {
      background-color: var(--primary-dark);
      color: white;
      text-align: left;
      padding: 15px;
    }

    td { padding: 12px 15px; border-bottom: 1px solid #eee; }

    /* Risk Badges */
    .risk-badge {
      padding: 5px 10px;
      border-radius: 20px;
      font-size: 12px;
      font-weight: bold;
      color: white;
      text-transform: uppercase;
    }

    .HIGH { background-color: var(--danger); }
    .MEDIUM { background-color: var(--warning); }
    .LOW { background-color: var(--success); }

    .loading-text { display: none; color: var(--accent-blue); font-weight: bold; }
  </style>
</head>
<body>

<div class="container">
  <h2>Transaction Analysis Dashboard</h2>
```

```
<div class="upload-card">
  <form id="uploadForm">
    <input type="file" id="fileInput" accept=".csv" required>
    <button type="submit" id="btnText">Run Analysis</button>
  </form>
  <p id="loading" class="loading-text">Analyzing patterns... please wait.</p>
</div>

<div class="summary-container">
  <div class="stat-card">
    <h3>Total Analyzed</h3>
    <span id="total">0</span>
  </div>
  <div class="stat-card">
    <h3>Fraud Suspected</h3>
    <span id="fraud">0</span>
  </div>
  <div class="stat-card">
    <h3>Normal</h3>
    <span id="normal">0</span>
  </div>
</div>

<h3>Transaction Detail</h3>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Amount</th>
```

```
            <th>Fraud Probability</th>
            <th>Result</th>
            <th>Risk Level</th>
          </tr>
        </thead>
        <tbody id="tableBody"></tbody>
    </table>
</div>

<script>
document.getElementById("uploadForm").addEventListener("submit", function(e) {
    e.preventDefault();

    const file = document.getElementById("fileInput").files[0];
    const loading = document.getElementById("loading");
    const btnText = document.getElementById("btnText");

    loading.style.display = "block";
    btnText.disabled = true;

    let formData = new FormData();
    formData.append("file", file);

    fetch("/analyze", {
        method: "POST",
        body: formData
    })
    .then(res => res.json())
    .then(data => {
        loading.style.display = "none";
```

```
    btnText.disabled = false;

    if(data.error) {
        alert(data.error);
        return;
    }

    // Summary Update
    document.getElementById("total").innerText = data.summary.total;
    document.getElementById("fraud").innerText = data.summary.fraud;
    document.getElementById("normal").innerText = data.summary.normal;

    // Table Update
    let table = document.getElementById("tableBody");
    table.innerHTML = "";

    data.results.forEach(row => {
        let tr = `
            <tr>
                <td>#${row.id}</td>
                <td>$${row.amount.toLocaleString()}</td>
                <td>${row.fraud_probability}%</td>
                <td>${row.prediction === 1 ? '🚩 Flagged' : '✅ Verified'}</td>
                <td><span class="risk-badge ${row.risk}">${row.risk}</span></td>
            </tr>
        `;
        table.innerHTML += tr;
    });
})
```

```
  .catch(err => {
    console.error(err);
    loading.style.display = "none";
    btnText.disabled = false;
  });
});
</script>

</body>
</html>
```

## OUTPUT